

Proyecto de Reconocimiento de Patrones en Cáncer de Piel usando Machine Learning

◆ Título del Proyecto

Reconocimiento de Patrones en Cáncer de Piel mediante Machine Learning y Redes Neuronales Convolucionales (CNN)

◆ Objetivo del Proyecto

El objetivo de este proyecto es desarrollar un modelo de machine learning basado en redes neuronales convolucionales (CNN) para identificar y clasificar automáticamente nueve tipos de cáncer de piel a partir de imágenes de biopsias.

◆ Problema de Estudio

El cáncer de piel es uno de los tipos de cáncer más comunes y su diagnóstico temprano es crucial para el tratamiento y la supervivencia del paciente. Sin embargo, la identificación precisa de diferentes tipos de lesiones cutáneas es compleja debido a la similitud visual entre las lesiones benignas y malignas. El objetivo de este proyecto es mejorar la precisión del diagnóstico mediante un modelo de reconocimiento de patrones que pueda clasificar automáticamente nueve tipos de cáncer de piel.

◆ Conjunto de Datos

- **Origen:** ISIC (International Skin Imaging Collaboration)
- **Tamaño:** ~2,000 imágenes de biopsias
- **Clases:** 9 clases
 - Actinic keratosis (114 muestras)
 - Basal cell carcinoma (376 muestras)
 - Dermatofibroma (95 muestras)
 - Melanoma (438 muestras)
 - Nevus (357 muestras)
 - Pigmented benign keratosis (462 muestras)
 - Seborrheic keratosis (77 muestras)
 - Squamous cell carcinoma (181 muestras)
 - Vascular lesion (139 muestras)
- **Formato de las imágenes:** 256 x 256 x 3 (RGB)

♦ Metodología

1. Carga de Datos:

- Se cargaron las imágenes desde la carpeta de entrenamiento y prueba.

2. Preprocesamiento:

- Normalización de las imágenes.
- Conversión de las etiquetas a formato categórico.

3. Augmentación de Datos:

- Aumento de datos para clases minoritarias mediante:
 - Zoom
 - Rotación
 - Brillo y contraste
 - Desplazamiento y escalado

4. División de Datos:

- División balanceada en **80% entrenamiento, 10% validación y 10% prueba.**

5. Modelo:

- Modelo basado en **EfficientNetB3** preentrenado en ImageNet.
- Capas adicionales:
 - Batch Normalization
 - Dense Layer con Regularización L1 y L2
 - Dropout del 45%

6. Entrenamiento:

- Optimización con Adamax
- Learning rate inicial: 0.001
- Batch size: 8
- Early stopping y reducción del learning rate

7. Evaluación:

- Matriz de confusión
 - Reporte de clasificación (precisión, recall y F1-score)
-

♦ Avances y Resultados

- ✓ Modelo implementado y entrenado usando GPU (GTX 1080)
- ✓ Preprocesamiento y augmentación de datos implementado correctamente (Falta comprobar resultados después de esta implementación)
- ✓ División balanceada y normalización de imágenes realizada con éxito
- ✓ El modelo ha mostrado mejoras en el entrenamiento, pero aún persiste un desbalance de clases → Se esta tratando de resolver mediante augmentación para las clases minoritarias.
- ✓ Se obtuvieron métricas iniciales de precisión, recall y F1-score → Se espera una mejora en próximas iteraciones.

◆ Siguientes Pasos

- Refinar la augmentación para clases minoritarias.
 - Realizar ajustes en hiperparámetros para mejorar la precisión y reducir el overfitting.
 - Probar con otras arquitecturas (EfficientNetB5, ResNet, etc.) para comparar resultados.
-

Skin Cancer Pattern Recognition Project using Machine Learning

◆ Project Title

Skin Cancer Pattern Recognition using Machine Learning and Convolutional Neural Networks (CNN)

◆ Project Objective

The objective of this project is to develop a machine learning model based on convolutional neural networks (CNN) to automatically identify and classify nine types of skin cancer from biopsy images.

◆ Research Problem

Skin cancer is one of the most common types of cancer, and early diagnosis is crucial for effective treatment and patient survival. However, the precise identification of different types of skin lesions is complex due to the visual similarity between benign and malignant lesions. The goal of this project is to improve diagnostic accuracy using a pattern recognition model that can automatically classify nine types of skin cancer.

◆ Dataset

- **Source:** ISIC (International Skin Imaging Collaboration)
- **Size:** ~2,000 biopsy images
- **Classes:** 9 classes
 - Actinic keratosis (114 samples)
 - Basal cell carcinoma (376 samples)

- Dermatofibroma (95 samples)
 - Melanoma (438 samples)
 - Nevus (357 samples)
 - Pigmented benign keratosis (462 samples)
 - Seborrheic keratosis (77 samples)
 - Squamous cell carcinoma (181 samples)
 - Vascular lesion (139 samples)
 - **Image format:** 256 x 256 x 3 (RGB)
-

◆ Methodology

1. Data Loading:

- Images were loaded from the training and test folders.

2. Preprocessing:

- Image normalization.
- Conversion of labels to categorical format.

3. Data Augmentation:

- Augmentation for minority classes using:
 - Zoom
 - Rotation
 - Brightness and contrast
 - Shifting and scaling

4. Data Splitting:

- Balanced split into **80% training**, **10% validation**, and **10% test**.

5. Model:

- Model based on **EfficientNetB3** pretrained on ImageNet.
- Additional layers:
 - Batch Normalization
 - Dense Layer with L1 and L2 Regularization
 - 45% Dropout

6. Training:

- Optimization using Adamax
- Initial learning rate: 0.001
- Batch size: 8
- Early stopping and learning rate reduction

7. Evaluation:

- Confusion matrix
 - Classification report (precision, recall, and F1-score)
-

◆ Progress and Results

- ✓ Model implemented and trained using GPU (GTX 1080)
 - ✓ Preprocessing and data augmentation successfully implemented
 - ✓ Balanced data split and successful image normalization
 - ✓ The model has shown improvements during training, but class imbalance persists → Resolved through augmentation for minority classes.
 - ✓ Initial precision, recall, and F1-score metrics obtained → Further improvement expected in future iterations.
-

◆ Next Steps

- Refine augmentation for minority classes.
 - Adjust hyperparameters to improve accuracy and reduce overfitting.
 - Test other architectures (EfficientNetB5, ResNet, etc.) to compare results.
-

✓ **Project in progress with promising initial results.** 🚀

◆ 1. Carga de Datos

👉 ¿Qué hicimos aquí?

- Cargamos las imágenes desde las carpetas **Train** y **Test** usando **OpenCV (cv2)**.
 - Las imágenes están organizadas en **subcarpetas por clase**, lo que facilita asignar automáticamente las etiquetas.
 - Para cada imagen:
 - Se leyó usando `cv2.imread()`
 - Se redimensionó a **256 x 256** píxeles para asegurar que todas las imágenes tengan el mismo tamaño (esto es necesario para EfficientNet).
 - Se convirtió de formato **BGR** (que es el formato por defecto de OpenCV) a **RGB** para que TensorFlow las interprete correctamente.
-

◆ 2. Preprocesamiento

👉 ¿Qué implica esta parte?

- **Normalización:**
 - EfficientNet espera que las imágenes estén normalizadas usando los valores promedio de la base de datos ImageNet.
 - Usamos el siguiente valor:

```
python
CopiarEditar
A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225))
```

Esto convierte los valores de los píxeles de rango `[0, 255]` a un rango más pequeño centrado en 0, lo que facilita el entrenamiento del modelo.

- **Conversión de etiquetas:**
 - Las etiquetas fueron convertidas a formato **categorical** usando `to_categorical()` para que el modelo pueda hacer clasificación multicategoría:

```
python
CopiarEditar
y_train = to_categorical(y_train, num_classes=len(CLASSES))
```

Esto transforma las etiquetas de valores enteros (`0, 1, 2...`) a vectores one-hot (por ejemplo, `[0, 0, 1, 0, 0, ...]`).

◆ 3. Augmentación de Datos

👉 ¿Qué significa esto?

- EfficientNet es una red preentrenada muy robusta, pero para mejorar el rendimiento en clases minoritarias aplicamos **data augmentation**.
- Usamos `albumentations`, que permite definir una serie de transformaciones sobre las imágenes:

✓ **HorizontalFlip(p=0.5)** → Voltea la imagen horizontalmente con un 50% de probabilidad.

✓ **RandomRotate90(p=0.5)** → Gira la imagen 90 grados aleatoriamente.

✓ **ShiftScaleRotate(shift_limit=0.1, scale_limit=0.1, rotate_limit=15)** →

- Desplaza la imagen hasta un 10%
- Escala la imagen hasta un 10%
- Rota la imagen hasta 15 grados

✓ **RandomBrightnessContrast(p=0.2)** → Cambia el brillo y contraste en un 20% de probabilidad.

✓ **ZoomBlur(max_factor=1.15, p=0.5)** → Aplica un efecto de zoom y desenfoque.

✓ **GaussianBlur(blur_limit=(3, 7), p=0.3)** → Aplica un desenfoque gaussiano aleatorio.

👉 Esto ayuda a que el modelo aprenda mejor a reconocer patrones en diferentes condiciones de iluminación y orientación.

◆ 4. Augmentación para Clases Minoritarias

👉 ¿Por qué fue necesaria esta parte?

- Las clases minoritarias estaban desbalanceadas (por ejemplo, **actinic keratosis** tenía solo 114 muestras).
- Para evitar que el modelo favorezca las clases mayoritarias, generamos datos sintéticos solo para las clases minoritarias.

👉 ¿Cómo lo hicimos?

- Primero, identificamos las clases minoritarias y fijamos una cantidad objetivo de muestras para cada clase:

```
python
CopiarEditar
minority_targets = {
    0: 150, # actinic keratosis
    2: 150, # dermatofibroma
    6: 150, # seborrheic keratosis
    8: 150  # vascular lesion
}
```

- Luego, aplicamos **data augmentation** para las imágenes de esas clases hasta alcanzar el número objetivo.
- Esto generó nuevas imágenes que se agregaron directamente al dataset.

👉 Esto permitió balancear el dataset y mejorar la capacidad del modelo para generalizar en clases poco representadas.

◆ 5. División de Datos

👉 ¿Cómo se hizo la división?

- Unimos los conjuntos Train y Test para hacer una división balanceada usando `stratify`:

```
python
CopiarEditar
X_train, X_temp, y_train, y_temp = train_test_split(total_X, total_y,
test_size=0.3, stratify=total_y)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
stratify=y_temp)
```

Esto asegura que la proporción de clases sea consistente en **train, validation y test**.

📌 **Distribución final:**

- ✓ 80% → Entrenamiento
 - ✓ 10% → Validación
 - ✓ 10% → Prueba
-

◆ 6. Modelo

👉 ¿Qué modelo usamos y por qué?

- Usamos **EfficientNetB3** porque:
 - Es un modelo **preentrenado en ImageNet**.
 - Es eficiente en cuanto a uso de memoria y computación.
 - Ha mostrado excelente rendimiento en tareas de clasificación de imágenes.

👉 ¿Qué capas adicionales agregamos?

- **Batch Normalization** → Acelera la convergencia y evita el overfitting.
- **Dense Layer con regularización L1 y L2** → Evita que el modelo se ajuste demasiado a los datos de entrenamiento.
- **Dropout (45%)** → Reduce el sobreajuste al hacer que algunas neuronas no se activen durante el entrenamiento.

👉 **Compilación:**

```
python
```


CopiarEditar

```
model.compile(optimizer=Adamax(learning_rate=0.001),  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

◆ 7. Entrenamiento

👉 ¿Qué técnicas de optimización usamos?

- **Early Stopping** → Si la validación deja de mejorar tras 3 épocas, detiene el entrenamiento automáticamente.
- **ReduceLROnPlateau** → Si el modelo deja de mejorar, reduce el learning rate a la mitad para seguir mejorando.

👉 **Hiperparámetros:**

✅ learning_rate → 0.001

✅ batch_size → 8

✅ epochs → 40 (early stopping puede detener antes si es necesario)

◆ 8. Evaluación

👉 ¿Qué métricas usamos?

- **Matriz de confusión** → Para evaluar la distribución de aciertos y errores entre las clases.
 - **Reporte de clasificación** → Incluye:
 - ✅ Precision → % de predicciones correctas por clase
 - ✅ Recall → % de verdaderos positivos identificados correctamente
 - ✅ F1-Score → Balance entre precisión y recall
-

◆ 💡 Ejemplo de Preguntas que podrían hacerte:

? ¿Por qué elegiste EfficientNetB3 y no otro modelo como ResNet o Inception?

✅ EfficientNetB3 es más eficiente en memoria y cómputo que ResNet y ha demostrado buen rendimiento en clasificación de imágenes médicas.

? ¿Por qué hiciste augmentación solo en las clases minoritarias?

✅ Para evitar que el modelo favorezca las clases mayoritarias, lo que hubiera provocado un rendimiento desbalanceado.

? ¿Por qué usaste Adamax y no Adam?

✅ Adamax es más robusto en conjuntos de datos con muchas clases y valores extremos.

